Linnæus University
Sweden

Lab Assignment 2

LEGv8 Assembly Programming

Computer Technology 1

October 17, 2026



Author: Andres Rodriguez
Examiner: Thomas
Semester: HT25
Discipline: Exchange Erasmus Student
Course code: 1dt301

Task 1

Assembly code

Listing 1: Hello world + counter program

```
.syntax unified
.cpu cortex-m0plus
.thumb

.thumb_func
.global main

main:
    MOVS R7, #100
    BL stdio_init_all

loop:
    LDR R0, =helloworld
    MOV R1, R7
    BL printf

    LDR R0, =1000
    BL sleep_ms

    MOVS R6, #0
    SUBS R7, #1
    CMP R7, R6
    BNE loop
    MOVS R7, #100
```

```
   B loop


.data
   .align 4
helloworld:
   .asciz "Hello World %d\n"
```

Comments

The code for the hello world adding a counter with 1 second of delay between messages, we just load the ascii string into r0, then we put the counter number into r1 and we use printf("hello world", int counter number) from sdk to print what we wanted, then we just subtract 1 from the counter and checked if we have reached 0 to restart the counter and loop forever.

Task 2

Assembly code

Listing 2: Traffic light assembly program

```
.syntax unified
.cpu cortex-m0plus
.thumb


.global mainS
```

```
.type mainS, %function
.align 2

.equ LED_GREEN, 0
.equ LED_YELLOW, 1
.equ LED_RED, 2

.equ GPIO_OUT, 1

mainS:
   push {lr}

   movs r0, #LED_GREEN
   movs r1, #GPIO_OUT
   bl link_gpio_set_dir

   movs r0, #LED_YELLOW
   movs r1, #GPIO_OUT
   bl link_gpio_set_dir

   movs r0, #LED_RED
   movs r1, #GPIO_OUT
   bl link_gpio_set_dir

traffic_loop:
   movs r0, #LED_RED
   movs r1, #1
   bl link_gpio_put
```

```
movs r0, #LED_YELLOW
movs r1, #0
bl link_gpio_put

movs r0, #LED_GREEN
movs r1, #0
bl link_gpio_put

ldr r0, =5000
bl sleep_ms

movs r0, #LED_RED
movs r1, #0
bl link_gpio_put

movs r0, #LED_YELLOW
movs r1, #1
bl link_gpio_put

movs r0, #LED_GREEN
movs r1, #0
bl link_gpio_put

ldr r0, =5000
bl sleep_ms

movs r0, #LED_RED
```

```
    movs r1, #0
    bl link_gpio_put

    movs r0, #LED_YELLOW
    movs r1, #0
    bl link_gpio_put

    movs r0, #LED_GREEN
    movs r1, #1
    bl link_gpio_put

    ldr r0, =5000
    bl sleep_ms

    b traffic_loop

    pop {pc}
```

Comments

This program controls a traffic light sequence when 3 LEDs are connected to GP0, GP1, and GP2.After initializing them with gpioSetDir as outputs (GPIOOUT), we create a simple loop where we turn on 1 LED at each stage of the cycle while turning off the others. We use a 5-second delay, and then we repeat the process by turning off the active LED and turning on the next one in sequence indefinitely. It can be seen that every time we call a function (bl functionName) we put the parameters of the function into r0,r1,r2..., and we push into the stack the LR and at the end we pop pc back to avoid problems after returning from a function.

Task 3

Assembly code

Listing 3: 3 bits counter assembly program

```
.syntax unified
.cpu cortex-m0plus
.thumb

.global asm_main
.type asm_main, %function
.align 2

.equ BIT0, 0 @LED representing bit 0
.equ BIT1, 1 @LED representing bit 1
.equ BIT2, 2 @LED representing bit 2

.equ GPIO_OUT, 1

asm_main:
   push {lr}

   movs r0, #BIT0
   movs r1, #GPIO_OUT
   bl link_gpio_set_dir

   movs r0, #BIT1
   movs r1, #GPIO_OUT
   bl link_gpio_set_dir
```

```
    movs r0, #BIT2
    movs r1, #GPIO_OUT
    bl link_gpio_set_dir

    movs r4, #0
count_loop:
    movs r0, #BIT0
    movs r1, r4
    movs r5, #1
    ands r1, r5
    bl link_gpio_put

    movs r0, #BIT1
    movs r1, r4
    lsrs r1, r1, #1
    movs r5, #1
    ands r1, r5
    bl link_gpio_put

    movs r0, #BIT2
    movs r1, r4
    lsrs r1, r1, #2
    movs r5, #1
    ands r1, r5
    bl link_gpio_put

    ldr r0, =1000
```

```
    bl sleep_ms


    adds r4, r4, #1
    movs r5, #8
    cmp r4, r5
    blt count_loop
    movs r4, #0
    b count_loop


    pop {pc}
```

Comments

This program implements a binary counter sequence using LEDs connected to GPIO pins (from GP0 to GP3). After initializing the pins as outputs (GPIOOUT) and setting up the system, we create a loop that increments a counter register on each iteration. The binary value of the counter is sent directly to the GPIO pins to display the corresponding count in binary. We use a delay (such as 1 second) between counts to make the sequence visible, resetting or wrapping around the counter when it reaches its maximum value, and repeating the process indefinitely. To transform a decimal number bounded between 0 and 7, to 3 binary outputs we just do a logical AND operation for the first bit, a logical AND after shifting to the right 1 position for the second bit and a logical AND operation shifting 2 positions to the right for the 3rd bit. As shown, every time we call a function (bl functionName), we pass its arguments through registers r0, r1, r2..., and we push lr onto the stack at the start and pop pc at the end to ensure proper execution flow when returning from subroutine calls.

Task 4

Assembly code

Listing 4: 7 led display, 0 to 9 and back to 0 counter

```
.syntax unified
.cpu cortex-m0plus
.thumb

.global asm_main
.type asm_main, %function
.align 2

asm_main:
   push {lr}
   bl link_init_display

infinite_loop:
   ldr r0,=0b01111101
   bl link_gpio_put_all
   ldr r0,=1000
   bl sleep_ms

   ldr r0,=0b01000100
   bl link_gpio_put_all
   ldr r0,=1000
   bl sleep_ms

   ldr r0,=0b01111010
```

```
bl link_gpio_put_all
ldr r0,=1000
bl sleep_ms

ldr r0, =0b01101110
bl link_gpio_put_all
ldr r0,=1000
bl sleep_ms

ldr r0,=0b01000111
bl link_gpio_put_all
ldr r0,=1000
bl sleep_ms

ldr r0,=0b00101111
bl link_gpio_put_all
ldr r0,=1000
bl sleep_ms

ldr r0,=0b00111111
bl link_gpio_put_all
ldr r0,=1000
bl sleep_ms

ldr r0,=0b01100110
bl link_gpio_put_all
ldr r0, =1000
bl sleep_ms
```

```
ldr r0,=0b01111111
bl link_gpio_put_all
ldr r0,=1000
bl sleep_ms

ldr r0,=0b01101111
bl link_gpio_put_all
ldr r0,=1000
bl sleep_ms

//Back to zero


ldr r0,=0b01111111
bl link_gpio_put_all
ldr r0,=1000
bl sleep_ms

ldr r0,=0b01100110
bl link_gpio_put_all
ldr r0, =1000
bl sleep_ms

ldr r0,=0b00111111
bl link_gpio_put_all
ldr r0,=1000
bl sleep_ms
```

```
ldr r0,=0b00101111
bl link_gpio_put_all
ldr r0,=1000
bl sleep_ms

ldr r0,=0b01000111
bl link_gpio_put_all
ldr r0,=1000
bl sleep_ms

ldr r0, =0b01101110
bl link_gpio_put_all
ldr r0,=1000
bl sleep_ms

ldr r0,=0b01111010
bl link_gpio_put_all
ldr r0,=1000
bl sleep_ms

ldr r0,=0b01000100
bl link_gpio_put_all
ldr r0,=1000
bl sleep_ms


b infinite_loop
```

```
    pop {pc}
```

Comments

This program controls a 7-segment display counter sequence when the display segments are connected to GPIO pins GP0 to GP7. After initializing these pins as outputs, we create a loop that sequentially outputs specific binary bitmasks formatted as `0b(DP)BAEDCGF` (where each letter represents the corresponding segment or decimal point to turn on) to display the digits using `link_gpio_put_all`. We use a delay (such as 1 second via `sleep_ms`) after displaying each number, and then we repeat the counting cycle indefinitely. As with previous functions, parameters are passed via registers (`r0`, `r1`, etc.), and function execution state is safely preserved using `push {lr}` and `pop {pc}`.

Few comments about the programming environment

While the conventional approach for RP2040 development involves running a Linux Virtual Machine, I opted for a fully native Windows configuration. This environment integrates the GNU Arm Embedded Toolchain, CMake, and VS Code directly on Windows.

The only limitation encountered was that executing the assembly routines required a minimal C main() wrapper to initialize the runtime and call the external assembly entry point.

Apart from this, the build workflow remains practically identical to the standard Linux environment, it relies on `pico_sdk_import.cmake` to integrate the SDK, a `CMakeLists.txt` file to manage the various project subdirectories, and individual `CMakeLists.txt` files to

define the specific source files (`main.c`, `main.S`) and explicitly enable features such as USB (`pico_enable_stdio_usb`).

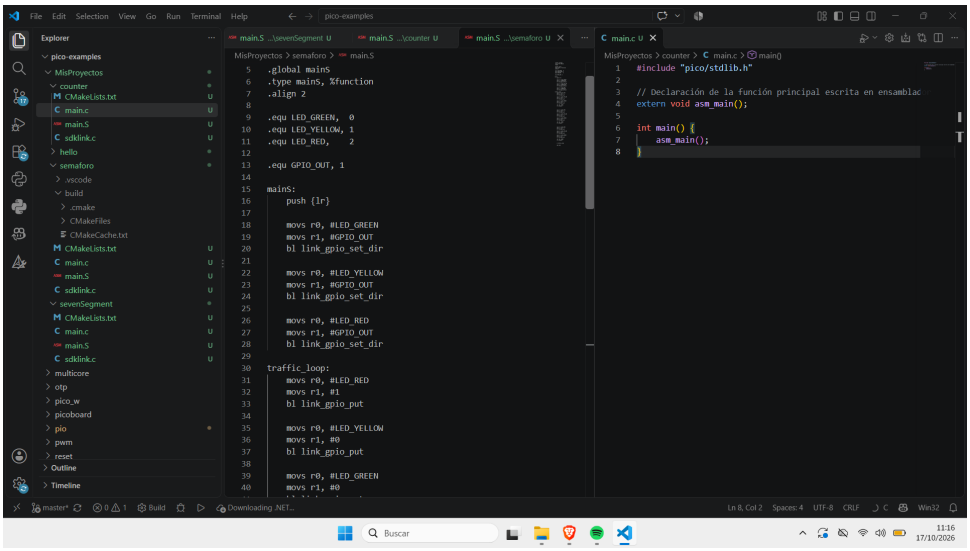


Figure 1: VS Code showing the C main wrapper and the assembly code for the 3 bits binary counter in Windows, CMakeLists.txt and sdk.c are also visible, apart from that at the left bar CMake is visible